

Sustainable Software Engineering

Apache JMeter

What are we going to do today?

Write simple performance test for Web Applications using Apache JMeter

<https://jmeter.apache.org/>

Useful Links

Apache JMeter

<https://github.com/apache/jmeter>

Performance testing

Performance Testing: the process of systematically evaluating performance features (e.g., execution time, throughput) of computing resources (e.g., CPU, memory) and software applications to find performance regressions

Load Testing: performance testing activity executing an artificially generated increasing workload against the system under test to verify its stability under normal load conditions

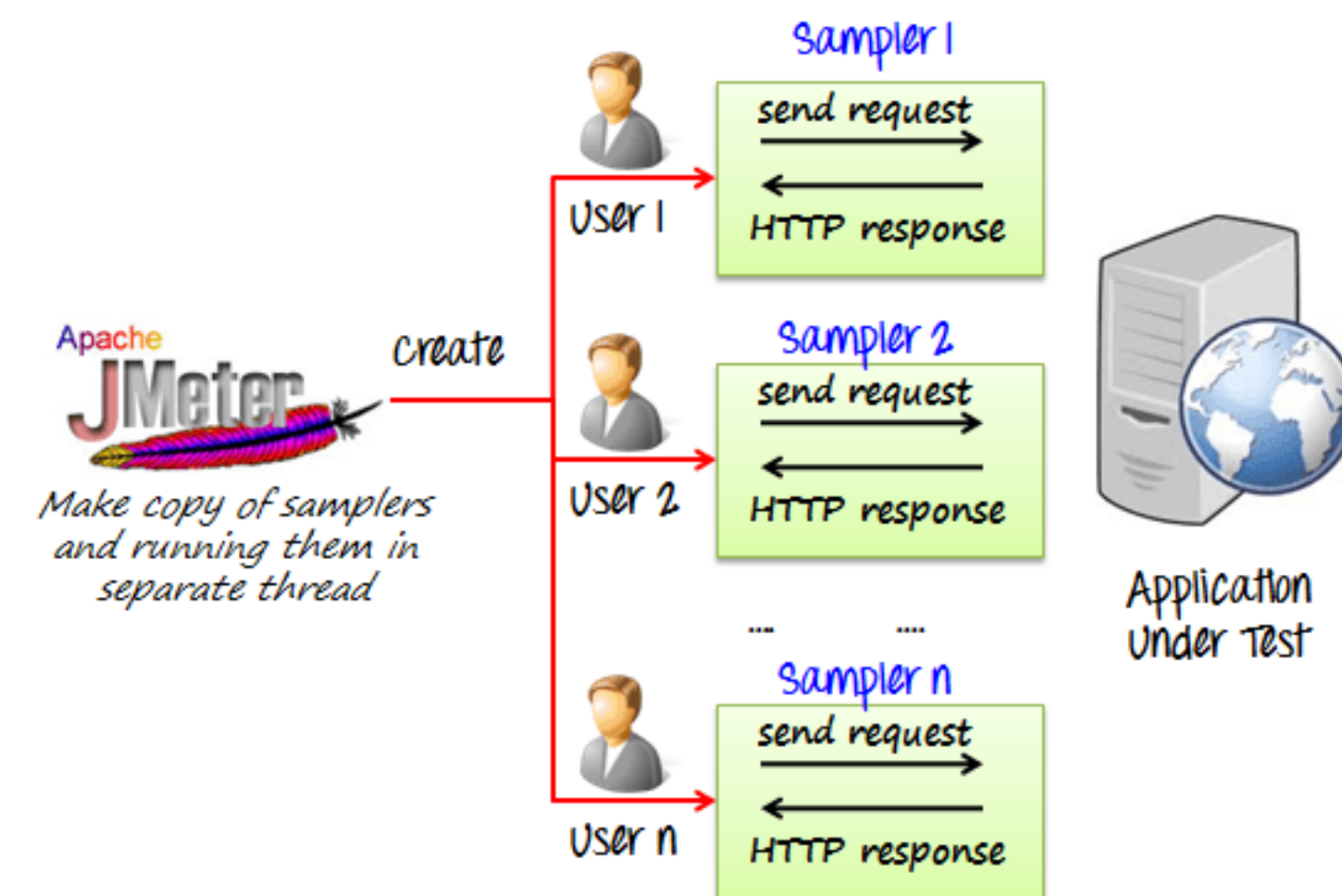
Stress Testing: performance testing activity executing an artificially generated workload against the system under test to determine its stability under beyond-normal load conditions, and to assess how it recovers after breakdowns

Spike Testing: performance testing activity executing a rapidly and significantly increasing and decreasing workload against the system under test to observe how the system reacts and recovers to sudden traffic spike

Soak Testing: performance testing activity executing an artificially moderate generated workload against the system under test for an extended period of time, to identify degradations appearing only over long-term operation

What Is Apache JMeter?

Apache JMeter: an open-source performance testing tool for web applications, and distributed systems



Apache JMeter is designed to **simulate realistic workloads** and **collect performance metrics** of the system under test (SUT)

The **results** obtained from the **target system** are **collected** and **described** using **graphs** and **tables**

Why Apache JMeter?

Objective: test the performance of the SUT replicating realistic production conditions

- **Being Open-source:** JMeter is employed by **practitioners** and **researchers**, and has a large **support community**
- **Workloads Simulations:** it allows to **simulate thousands of users** requesting the SUT, to test **stability** and **scalability**
- **Multi-threading Support:** each **user** is assigned to a single **thread**. A **thread pool** executing a test is a **Thread Group**
- **Multi-protocol Support:** JMeter supports many protocols (e.g., HTTP, HTTPS, SOAP, REST, JDBC), being very **flexible**
- **Intuitive GUI:** the ease of the GUI let **less expert developers** to produce **effective performance testing**
- **Detailed Metrics Support:** JMeter **collects** and **presents key metrics** like: **latency**, **throughput**, **error rates**
- **Scripting support:** **performance tests** and their **flows** can be **customized**, overcoming **preconfigured tests limits**
- **Caching and Cookies Support:** to store resources and manage sessions as in **production** it uses caching and cookies

The Main Components of Apache JMeter

- **Thread Groups:** the starting point, defining the number of simulated users, a **ramp-up time**, and number of iterations
- **Threads:** each thread is **single user independent** from all the **others**, executing its **Thread Group test flow**
- **Samplers:** they send the **requests**, each sampler is an **user operation** made during the test (e.g., login, send form)
- **Listeners:** they **collect** the **results** from the **Samplers**, show them in **graphs** or **tables**, and **save** them into output **files**
- **Pre-/Post-processors:** executed before and after a **Sampler** to **manipulate** or **analyze parameters** and **results**
- **Controllers:** they **control** the tests **logic**, like the **order** or **conditions** of **Samplers**, simulating **realistic scenarios**
- **Assertions:** allow the **developers** to **validate responses** received from the SUT